

INTERNSHIP PROGRAM

Problem Statement

Smarter alcohol detection system using STM32 that gives three levels of alert and logs every reading to a PC.

A. Introduction

Drunk driving is one of the major causes of road accidents in India, and timely detection of alcohol consumption in drivers can help prevent these accidents. Traffic police and checkpoint authorities currently rely on imported breathalyzer devices which are expensive and give only a pass or fail result with no data logging. A low cost microcontroller based alcohol detection system that can measure alcohol levels in a driver's breath, give multi-level alerts, and log every reading with the result would be a practical and affordable solution for roadside testing. This project builds such a system using STM32 microcontroller and MQ-3 alcohol sensor.

B. Problem Statement

Drunk driving checkpoints in India lack affordable and reliable alcohol detection tools that can provide graduated alert levels and maintain a record of test results. Current solutions either give a simple pass/fail output or are too expensive for widespread deployment. This project aims to build a microcontroller based alcohol detection prototype using STM32 and MQ-3 sensor that gives three levels of alert — safe, warning, and danger — and logs every reading to a PC for record keeping.

C. Market Review / Existing Solutions and Limitations

Several alcohol detection solutions exist in the market today. Digital breathalyzers used by traffic police provide a numerical BAC reading but cost between ₹5,000 to ₹50,000 and do not log data. Basic MQ-3 sensor based Arduino projects available as DIY kits give only a binary output — drunk or not drunk — with no alert levels or data recording. Smartphone based breathalyzer attachments are available but require a mobile app and are not suitable for official roadside testing. None of these solutions combine affordability, multi-

level detection, and data logging in a single embedded system. This project addresses all three gaps using an STM32 microcontroller with bare metal firmware.

D. References

1. Techatronic, "Alcohol Detector using Arduino and MQ3 Sensor," [Online]. Available: <https://techatronic.com/alcohol-detector-using-arduino-and-mq3-sensor/>
2. STMicroelectronics, "STM32F401xB/STM32F401xC/STM32F401xD/STM32F401xE Reference Manual (RM0368)," [Online]. Available: https://www.st.com/resource/en/reference_manual/rm0368.pdf
3. Zhengzhou Winsen Electronics, "MQ-3 Alcohol Gas Sensor Datasheet," [Online]. Available: <https://www.winsen-sensor.com/sensors/alcohol-sensor/mq-3.html>

SCOPE OF SOLUTION

1. Scope

This project covers the design and development of a microcontroller based alcohol detection prototype for roadside testing. The system detects alcohol in a driver's breath using the MQ-3 sensor and provides three levels of alert with data logging via UART to a PC.

2. Boundaries and Objectives

- Detects alcohol concentration in breath only — not blood alcohol level
- Works as a standalone prototype — not integrated into a vehicle
- Data logged to PC terminal during testing session only

3. Solution Aims

- Detect three levels: Safe, Warning, Danger
- Log every reading with result to PC
- Trigger buzzer and LED alerts based on level
- Use watchdog timer for system reliability

4. Constraints and Limitations

- MQ-3 sensor requires 2-3 minute warm-up time before accurate readings
- Sensor detects ethanol gas broadly — not medically certified for legal BAC measurement
- Readings may vary with humidity and temperature

5. Justification

STM32 is chosen over Arduino because it uses the same Cortex-M4 architecture found in automotive ECUs, making this project industry-relevant. Bare metal firmware without HAL ensures faster response and better understanding of hardware behavior.

REQUIRED COMPONENTS

1. Hardware Components and Their information

Component	Specification	Purpose
STM32 Nucleo F401RE	ARM Cortex-M4, 84MHz	Main microcontroller
MQ-3 Alcohol Sensor Module	Analog output, 5V	Detects alcohol in breath

Component	Specification	Purpose
Red LED	5mm, 2V	Danger level alert
Yellow LED	5mm, 2V	Warning level alert
Green LED	5mm, 2V	Safe level indicator
Buzzer	5V DC	Audio alert on danger
220Ω Resistors	1/4W	Current limiting for LEDs
Breadboard	840 points	Circuit assembly
Jumper Wires	M-M, M-F, F-F	Connections
USB A to Mini-B Cable	—	Power and programming

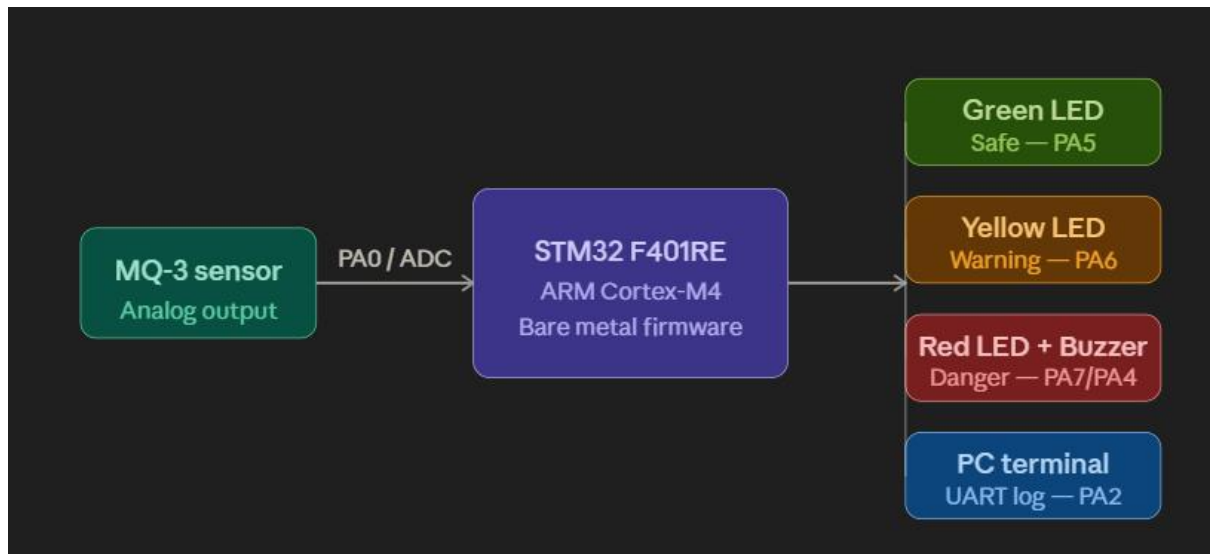
2.IDE

- STM32CubeIDE (Version 1.x) — free, downloaded from [st.com](https://www.st.com)

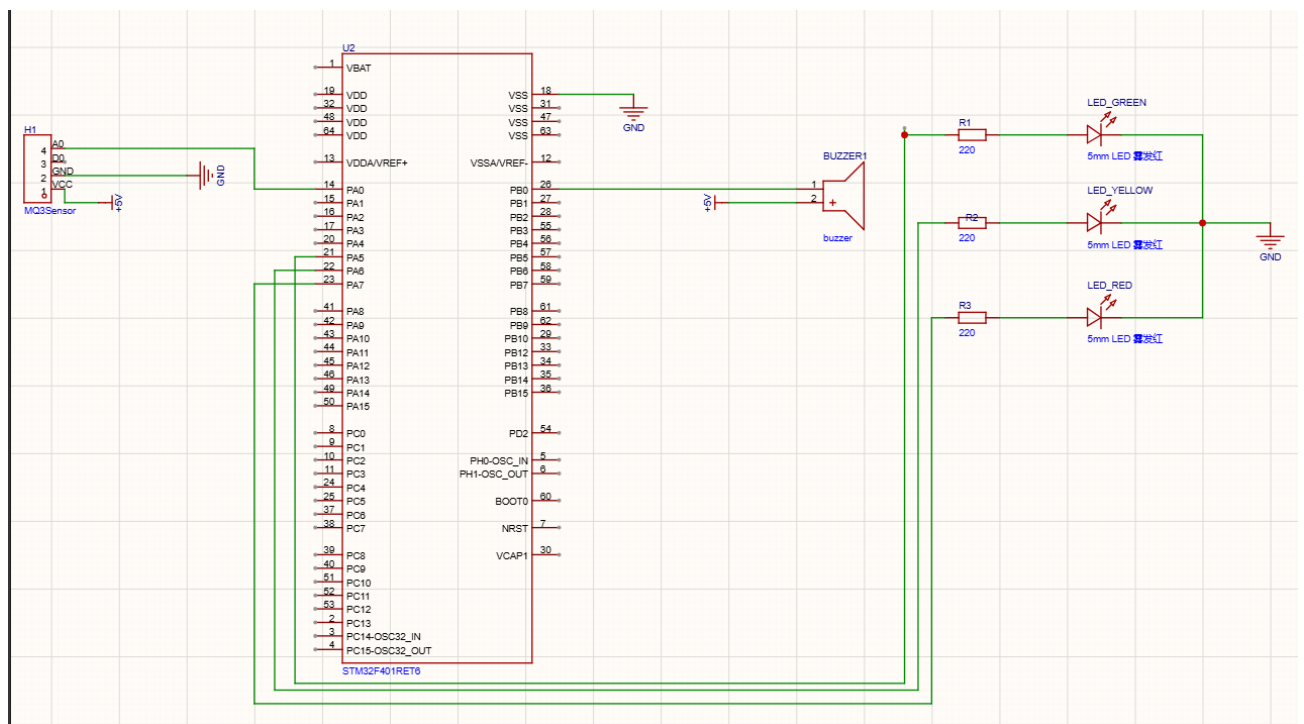
3.Software Tools and Frameworks

- STM32CubeIDE — firmware development and flashing
- EasyEDA — schematic design and Gerber file export
- STM32 RM0368 Reference Manual — register level programming reference
- GitHub — project submission and version control

BLOCK DIAGRAM

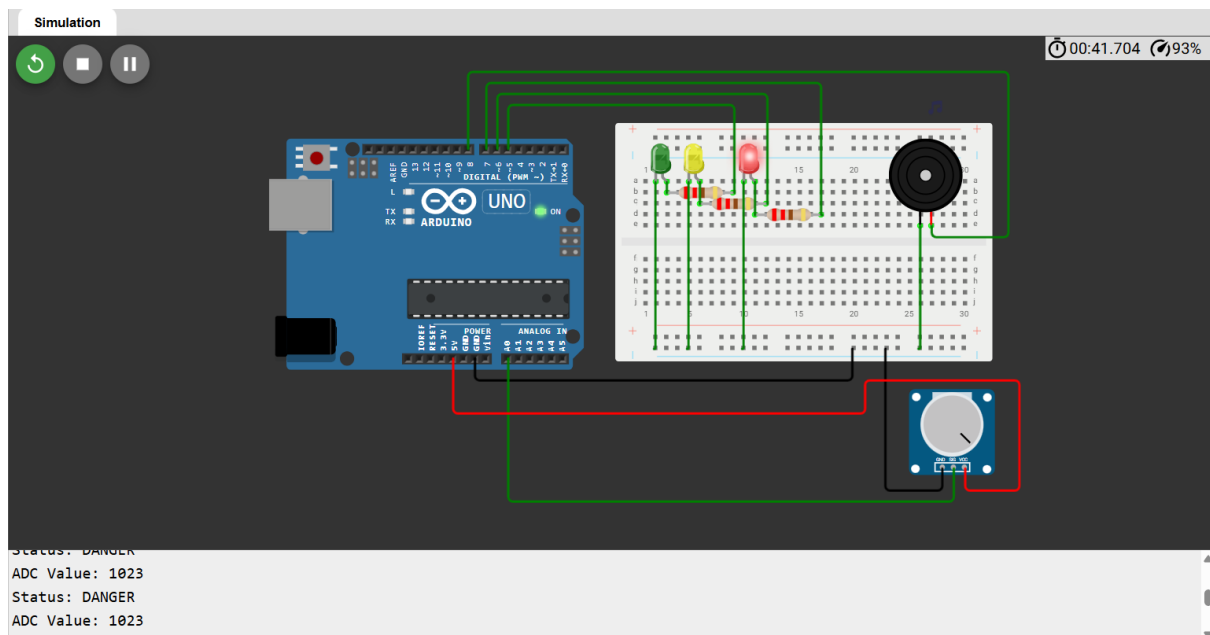


CIRCUIT DIAGRAM



The STM32F401RE is the main controller of this system. The MQ-3 sensor is powered by 5V and its output is connected to PA0, where the microcontroller reads the alcohol level as a number. Depending on how high that number is, it turns on one of the three LEDs connected to PA5, PA6, and PA7 through 220Ω resistors. If the level is dangerous, the buzzer connected to PB0 also turns on along with the red LED. All readings are sent to a PC through UART for logging.

SIMULATED CIRCUIT



The circuit was simulated using **Wokwi** (wokwi.com), a free online embedded systems simulator. Arduino UNO was used in place of STM32F401RE for simulation because Wokwi does not support the STM32F4 series. The circuit concept and logic remains identical — the potentiometer simulates the analog output of the MQ-3 alcohol sensor, and the same three-level detection logic (Safe, Warning, Danger) is implemented. The actual physical prototype uses STM32F401RE with bare metal firmware.

CODE FOR THE SOLUTION

```
1  /* USER CODE BEGIN Header */
2  /**
3   *
4   * @file      : main.c
5   * @brief     : Main program body
6   *
7   * @attention
8   *
9   * Copyright (c) 2026 STMicroelectronics.
10  * All rights reserved.
11  *
12  * This software is licensed under terms that can be found in the LICENSE file
13  * in the root directory of this software component.
14  * If no LICENSE file comes with this software, it is provided AS-IS.
15  *
16  *
17  */
18  /* USER CODE END Header */
19  /* Includes -----*/
20  #include "main.h"
21  #include <stdio.h>
22  #include <stdint.h>
23
24  /* Private includes -----*/
25  /* USER CODE BEGIN Includes */
26
27  /* USER CODE END Includes */
28
29  /* Private typedef -----*/
30  /* USER CODE BEGIN PTD */
31
32  /* USER CODE END PTD */
```

```
main.c X
51 void SystemClock_Config(void);
52 /* USER CODE BEGIN PFP */
53
54 /* USER CODE END PFP */
55
56 /* Private user code -----*/
57 /* USER CODE BEGIN 0 */
58 volatile uint32_t *const RCC_AHB1ENR = (volatile uint32_t*) 0x40023830;
59 volatile uint32_t *const RCC_APB2ENR = (volatile uint32_t*) 0x40023844;
60 volatile uint32_t *const RCC_APB1ENR = (volatile uint32_t*) 0x40023840;
61
62 volatile uint32_t *const GPIOA_MODER = (volatile uint32_t*) 0x40020000;
63 volatile uint32_t *const GPIOA_ODR = (volatile uint32_t*) 0x40020014;
64 volatile uint32_t *const GPIOA_AFRL = (volatile uint32_t*) 0x40020020;
65
66 volatile uint32_t *const GPIOB_MODER = (volatile uint32_t*) 0x40020400;
67 volatile uint32_t *const GPIOB_ODR = (volatile uint32_t*) 0x40020414;
68
69 volatile uint32_t *const ADC1_SR = (volatile uint32_t*) 0x40012000;
70 volatile uint32_t *const ADC1_CR1 = (volatile uint32_t*) 0x40012004;
71 volatile uint32_t *const ADC1_CR2 = (volatile uint32_t*) 0x40012008;
72 volatile uint32_t *const ADC1_SQR3 = (volatile uint32_t*) 0x4001203C;
73 volatile uint32_t *const ADC1_DR = (volatile uint32_t*) 0x4001204C;
74
75 volatile uint32_t *const USART2_SR = (volatile uint32_t*) 0x40004400;
76 volatile uint32_t *const USART2_DR = (volatile uint32_t*) 0x40004404;
77 volatile uint32_t *const USART2_BRR = (volatile uint32_t*) 0x40004408;
78 volatile uint32_t *const USART2_CR1 = (volatile uint32_t*) 0x4000440C;
79 void delay(volatile int count) {
80     while(count--);
81 }
82
```

```

82
83 void uart2_init(void) {
84     *RCC_APB1ENR |= (1U << 17);           // USART2 clock enable
85
86     *GPIOA_MODER &= ~(3U << 4);
87     *GPIOA_MODER |= (2U << 4);           // PA2 alternate function mode
88
89     *GPIOA_AFRL &= ~(0xF << 8);
90     *GPIOA_AFRL |= (7U << 8);           // AF7 = USART2 TX on PA2
91
92     *USART2_BRR = 0x0683;                // 9600 baud @ 16MHz
93     *USART2_CR1 |= (1U << 3);           // TE - transmitter enable
94     *USART2_CR1 |= (1U << 13);          // UE - USART enable
95 }
96
97 void uart2_send_char(char c) {
98     while(!(*USART2_SR & (1U << 7)));    // wait for TXE flag
99     *USART2_DR = c;
100 }
101
102 void uart2_send_string(char *str) {
103     while(*str) {
104         uart2_send_char(*str);
105         str++;
106     }
107 }
108
109 void uart2_send_number(uint16_t num) {
110     char buffer[6];
111     sprintf(buffer, "%d", num);
112     uart2_send_string(buffer);
113 }

```

```

121 /*
122 int main(void)
123 {
124
125     /* USER CODE BEGIN 1 */
126
127     /* USER CODE END 1 */
128
129     /* MCU Configuration-----*/
130
131     /* Reset of all peripherals, Initializes the Flash interface and the Systick. */
132     HAL_Init();
133
134     /* USER CODE BEGIN Init */
135
136     /* USER CODE END Init */
137
138     /* Configure the system clock */
139     SystemClock_Config();
140
141     /* USER CODE BEGIN SysInit */
142
143     /* USER CODE END SysInit */
144
145     /* Initialize all configured peripherals */
146     /* USER CODE BEGIN 2 */
147     *RCC_AHB1ENR |= (1U << 0);           // GPIOA clock enable
148     *RCC_AHB1ENR |= (1U << 1);           // GPIOB clock enable
149     *RCC_APB2ENR |= (1U << 8);           // ADC1 clock enable
150
151     *GPIOA_MODER |= (3U << 0);           // PA0 analog mode - MQ-3 AOUT
152

```


main.c ×

```
146 /* USER CODE BEGIN 2 */
147 *RCC_AHB1ENR |= (1U << 0); // GPIOA clock enable
148 *RCC_AHB1ENR |= (1U << 1); // GPIOB clock enable
149 *RCC_APB2ENR |= (1U << 8); // ADC1 clock enable
150
151 *GPIOA_MODER |= (3U << 0); // PA0 analog mode – MQ-3 AOUT
152
153 *GPIOA_MODER &= ~(3U<<10)|(3U<<12)|(3U<<14));
154 *GPIOA_MODER |= (1U<<10)|(1U<<12)|(1U<<14); // PA5,PA6,PA7 output
155
156 *GPIOB_MODER &= ~(3U<<0);
157 *GPIOB_MODER |= (1U<<0); // PB0 output – buzzer
158
159 *ADC1_SQR3 = 0; // channel 0 selected
160 *ADC1_CR2 |= (1U<<0); // ADC ON
161
162 uart2_init();
163
164 /* USER CODE END 2 */
165
166 /* Infinite loop */
167 /* USER CODE BEGIN WHILE */
168 while (1)
169 {
170 /* USER CODE END WHILE */
171
172 /* USER CODE BEGIN 3 */
173 *ADC1_CR2 |= (1U<<30); // start conversion
174 while(!(*ADC1_SR & (1U<<1))); // wait for EOC flag
175 uint16_t adc_val = *ADC1_DR; // read converted value
176
```

main.c ×

```
167 /* USER CODE BEGIN WHILE */
168 while (1)
169 {
170     /* USER CODE END WHILE */
171
172     /* USER CODE BEGIN 3 */
173     *ADC1_CR2 |= (1U<<30);           // start conversion
174     while(!(*ADC1_SR & (1U<<1)));    // wait for EOC flag
175     uint16_t adc_val = *ADC1_DR;     // read converted value
176
177     uart2_send_string("ADC: ");
178     uart2_send_number(adc_val);
179     uart2_send_string(" Status: ");
180
181     if (adc_val < 1700) {
182         *GPIOA_ODR |= (1U<<5);
183         *GPIOA_ODR &= ~(1U<<6);
184         *GPIOA_ODR &= ~(1U<<7);
185         *GPIOB_ODR |= (1U<<0);
186         uart2_send_string("SAFE\r\n");
187     }
188     else if (adc_val < 2200) {
189         *GPIOA_ODR &= ~(1U<<5);
190         *GPIOA_ODR |= (1U<<6);
191         *GPIOA_ODR &= ~(1U<<7);
192         *GPIOB_ODR |= (1U<<0);
193         uart2_send_string("WARNING\r\n");
194     }
195     else {
196         *GPIOA_ODR &= ~(1U<<5);
197         *GPIOA_ODR &= ~(1U<<6);
198         *GPIOA_ODR |= (1U<<7);
```

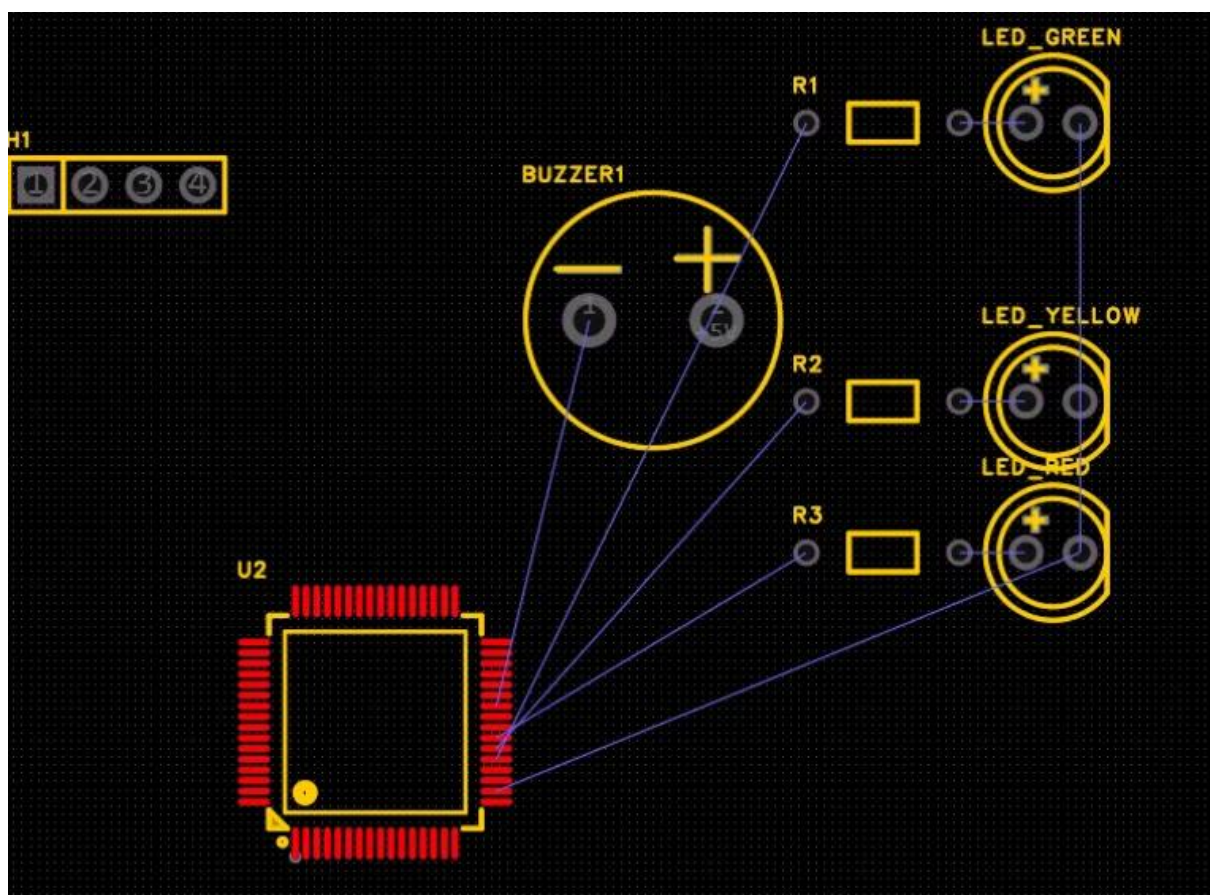
```
main.c ×
180
181     if (adc_val < 1700) {
182         *GPIOA_ODR |= (1U<<5);
183         *GPIOA_ODR &= ~(1U<<6);
184         *GPIOA_ODR &= ~(1U<<7);
185         *GPIOB_ODR |= (1U<<0);
186         uart2_send_string("SAFE\r\n");
187     }
188     else if (adc_val < 2200) {
189         *GPIOA_ODR &= ~(1U<<5);
190         *GPIOA_ODR |= (1U<<6);
191         *GPIOA_ODR &= ~(1U<<7);
192         *GPIOB_ODR |= (1U<<0);
193         uart2_send_string("WARNING\r\n");
194     }
195     else {
196         *GPIOA_ODR &= ~(1U<<5);
197         *GPIOA_ODR &= ~(1U<<6);
198         *GPIOA_ODR |= (1U<<7);
199         *GPIOB_ODR &= ~(1U<<0);
200         uart2_send_string("DANGER\r\n");
201     }
202     uart2_send_string("PB0=");
203     uart2_send_number((*GPIOB_ODR >> 0) & 1);
204     uart2_send_string("\r\n");
205
206
207     delay(500000);
208
209 }
210 /* USER CODE END 3 */
211 }
```

RESULTS

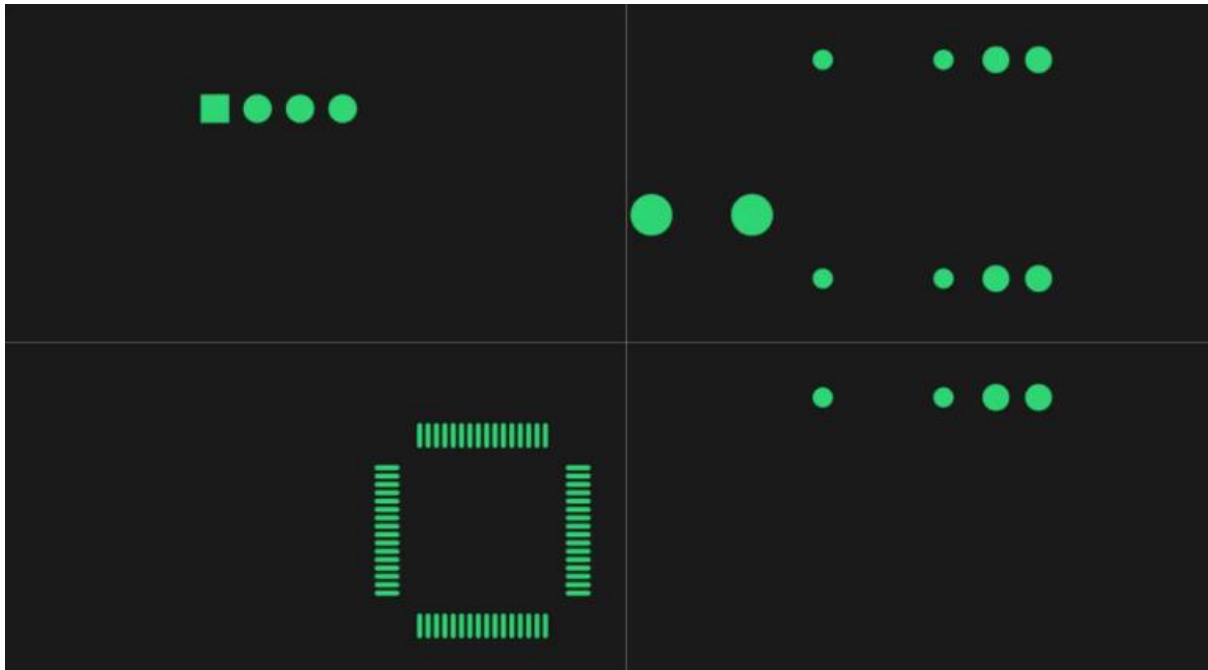
When the system is powered on, the MQ-3 sensor initially shows a higher ADC reading because the sensor's heating element needs time to stabilize. For this sensor unit, it took about 10 minutes to settle to a stable baseline reading. During this warm-up period, the LED may briefly show Red or Yellow even in clean air. Once stabilized, the system settles at its baseline reading, which corresponds to the Green (Safe) LED.

When alcohol vapor was introduced near the sensor (tested using an alcohol-based spray), the ADC value increased immediately, and the corresponding LED changed from Green to Yellow (Warning) and then Red (Danger) as the concentration increased. The buzzer's intensity also increased at the Danger level. All readings were logged live to the PC terminal via UART. All three states — Safe, Warning, and Danger — along with the live UART readings, are clearly shown in the demo video linked in the final section.

LAYOUT OF THE CIRCUIT



GERBER FILE



VIDEO OF THE DEMO (PROTOTYPE)

The video below shows the physical prototype working in real time. It captures the MQ-3 sensor warm-up period, the live ADC readings printed on the PC terminal via UART, and the LED transitioning through Safe (Green), Warning (Yellow), and Danger (Red) states as alcohol vapor is introduced near the sensor using a spray. The buzzer response at the Danger level is also shown.

Video Link:

<https://drive.google.com/file/d/1hpnVCpo7YsoebW96zgwUghl7gENcq0rz/view?usp=sharing>